CE 488/5588: Applied AI
Topic 13: Deep Neural Networks — Foundations & Architectures

Lecture Roadmap

1. Why Deep Learning Works Now
2. The Numerical Secrets of Training Deep Networks
3. Convolutional Neural Networks
4. Transformers
5. Diffusion Models
6. Comparing Architectures and Looking Ahead

From Classical Machine Learning to Deep Learning

Topics 1–12 emphasized classical machine learning: regression, classification, SVMs, Bayesian learning, clustering, and dimensionality reduction.

What Changes Here?

We now shift from models that depend heavily on hand-crafted features to models that learn internal representations directly from raw data.

This transition is the gateway to the post-2020 AI era: foundation models, generative models, and agentic systems.

Why Was Deep Learning Not Dominant Earlier?

Neural networks are not new.

- perceptrons date to the 1950s,
- backpropagation was formalized in the 1980s,
- multilayer perceptrons were already known before the current AI boom.

Historical Reality

For decades, deep networks were often too hard to train reliably, too computationally expensive, and too data-hungry for routine engineering use.

The 2012 Inflection Point

The ImageNet 2012 result is often treated as the symbolic turning point.

- AlexNet dramatically improved visual recognition performance,
- GPU-based training proved practical at scale,
- deep architectures suddenly became hard to ignore.

Source: Krizhevsky, Sutskever, & Hinton (NIPS 2012)

Three Forces Converged

Why Now?

Modern deep learning became practical because three forces converged: more data, more compute, and better training algorithms.

- **Data:** internet-scale corpora and benchmark datasets,
- **Compute:** GPUs and parallel linear algebra,
- **Algorithms:** activation, initialization, normalization, and optimization advances.

Why Training Is the Real Story

A deep network is not useful simply because it has many layers.

Core Claim

The real breakthrough was not the idea of stacking layers. The breakthrough was learning how to train deep stacks reliably.

This section explains the main numerical reasons that modern deep learning became practical.

Backpropagation Through Many Layers

For an early layer, the gradient depends on repeated application of the chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}} = \frac{\partial \mathcal{L}}{\partial \mathbf{a}^{(L)}} \cdot \frac{\partial \mathbf{a}^{(L)}}{\partial \mathbf{a}^{(L-1)}} \cdots \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{W}^{(1)}}.$$

If many of these terms are small, the product becomes extremely small. If many are large, the product can explode.

The Vanishing Gradient Problem

The sigmoid derivative never exceeds 0.25.

What Goes Wrong?

If each layer contributes a factor smaller than 1, then gradients decay exponentially as we move backward. Early layers receive almost no learning signal.

This explains why early neural networks were often shallow in practice.

Activation Functions and Gradient Flow

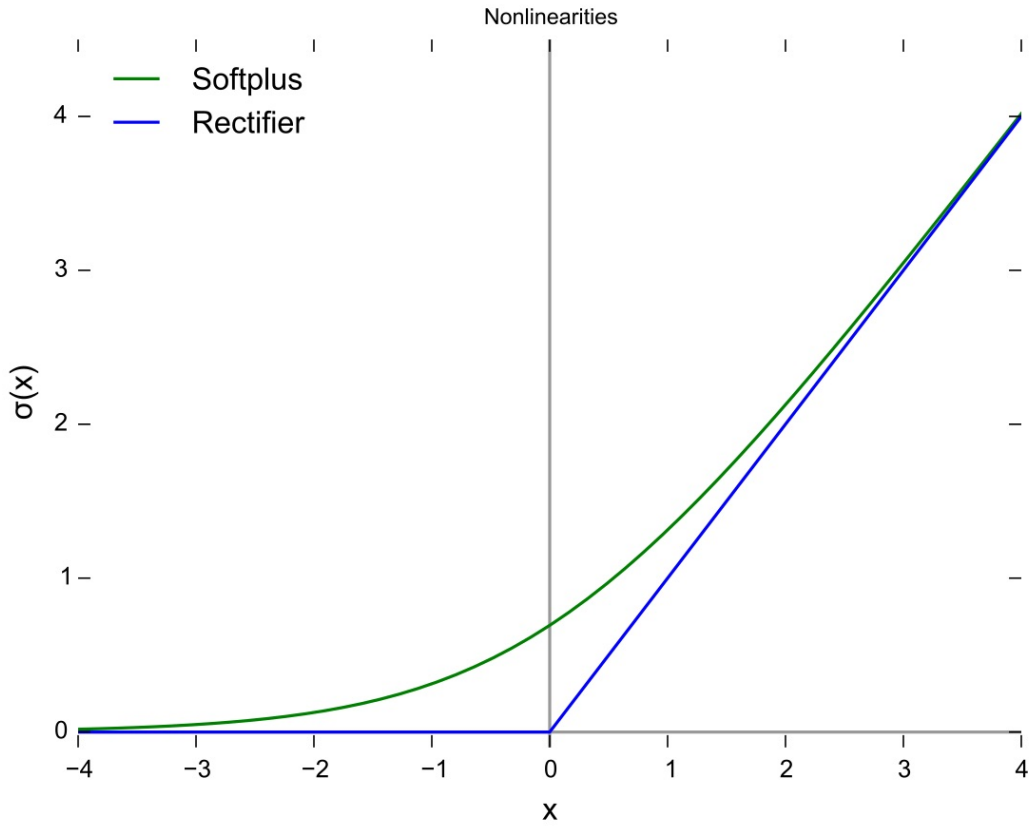

Figure: Activation functions commonly discussed in deep learning.

The engineering question is not only expressive power. It is also whether the activation supports stable gradient propagation.

Why ReLU Was So Important

ReLU is defined by

$$\mathrm{ReLU}(x) = \max(0, x).$$

Key Numerical Benefit

For positive inputs, the derivative is exactly 1. That prevents the repeated shrinking behavior that destroys gradients in sigmoid-style networks.

ReLU is also computationally cheap, which matters on large GPU workloads.

Initialization Matters from the First Step

Training can fail before it really starts if weights are initialized poorly.

Two Failure Modes

If initial weights are too large, activations and gradients explode. If they are too small, useful signals vanish before learning stabilizes.

Good initialization tries to preserve variance across layers.

Xavier and He Initialization

Two classic initialization rules are:

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right)$$

for Xavier/Glorot-style scaling, and

$$W_{ij} \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

for He initialization used with ReLU.

Interpretation

The network should start in a numerically reasonable operating regime before optimization even begins.

Batch Normalization: Why Normalization Helps

As upstream weights change, the distribution seen by downstream layers shifts during training.

Batch Normalization Idea

Normalize activations within a mini-batch, then allow the network to learn a rescaling and shift. This keeps the training dynamics better conditioned.

Source: Ioffe & Szegedy (ICML 2015)

Batch Normalization Equation

Batch normalization computes

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \qquad y_i = \gamma \hat{x}_i + \beta.$$

The first step standardizes the mini-batch. The second step restores flexibility so the network is not forced into one fixed normalized representation.

Practical Effects of Batch Normalization

Batch normalization often:

- stabilizes optimization,
- permits larger learning rates,
- reduces sensitivity to initialization, and
- introduces a mild regularizing effect through batch noise.

It was one of the main ingredients that made very deep networks routine rather than fragile.

Optimization Before Adam

Basic stochastic gradient descent uses

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \nabla_{\mathbf{w}} \mathcal{L}.$$

That looks simple, but in practice a fixed learning rate can be difficult to tune.

- Too small: training is painfully slow.
- Too large: optimization diverges or oscillates.

Momentum as a Physical Analogy

Momentum introduces a velocity term:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + (1 - \beta) \nabla_{\mathbf{w}} \mathcal{L}.$$

Physical Intuition

Momentum dampens noisy zig-zag motion and accelerates progress along directions where gradients keep pointing consistently downhill.

Adam and Adaptive Learning Rates

Adam tracks both the first and second moments of the gradient and uses them to adapt the step size for each parameter.

- first moment: direction and trend,
- second moment: gradient magnitude and variability,
- practical result: a more forgiving optimizer in large models.

Source: Kingma & Ba (Adam)

The Modern Training Recipe

The Numerical Secrets in One Slide

ReLU improves gradient flow. He initialization starts in a stable range. Batch normalization keeps activations well behaved. Adam makes optimization easier to tune.

Together, these changes transformed deep learning from a temperamental idea into a robust engineering workflow.

Why Images Need a Different Bias

Images are not generic vectors.

- nearby pixels are related,
- edges and textures are local,
- the same feature can appear in many locations.

Inductive Bias

CNNs encode locality and translation structure directly into the architecture.

From Hand-Crafted Features to Learned Features

Earlier computer vision relied on expert-designed descriptors such as:

- edges,
- gradients,
- corners,
- texture statistics.

CNNs replace manual feature design with filters whose useful patterns are learned from data.

The Convolution Operation

A small kernel slides across an image and computes local weighted sums:

$$(\mathbf{I} * \mathbf{K})_{i,j} = \sum_m \sum_n \mathbf{I}_{i+m,j+n} \mathbf{K}_{m,n}.$$

The kernel entries are trainable parameters, so the network discovers which local patterns matter.

Visualizing Convolution

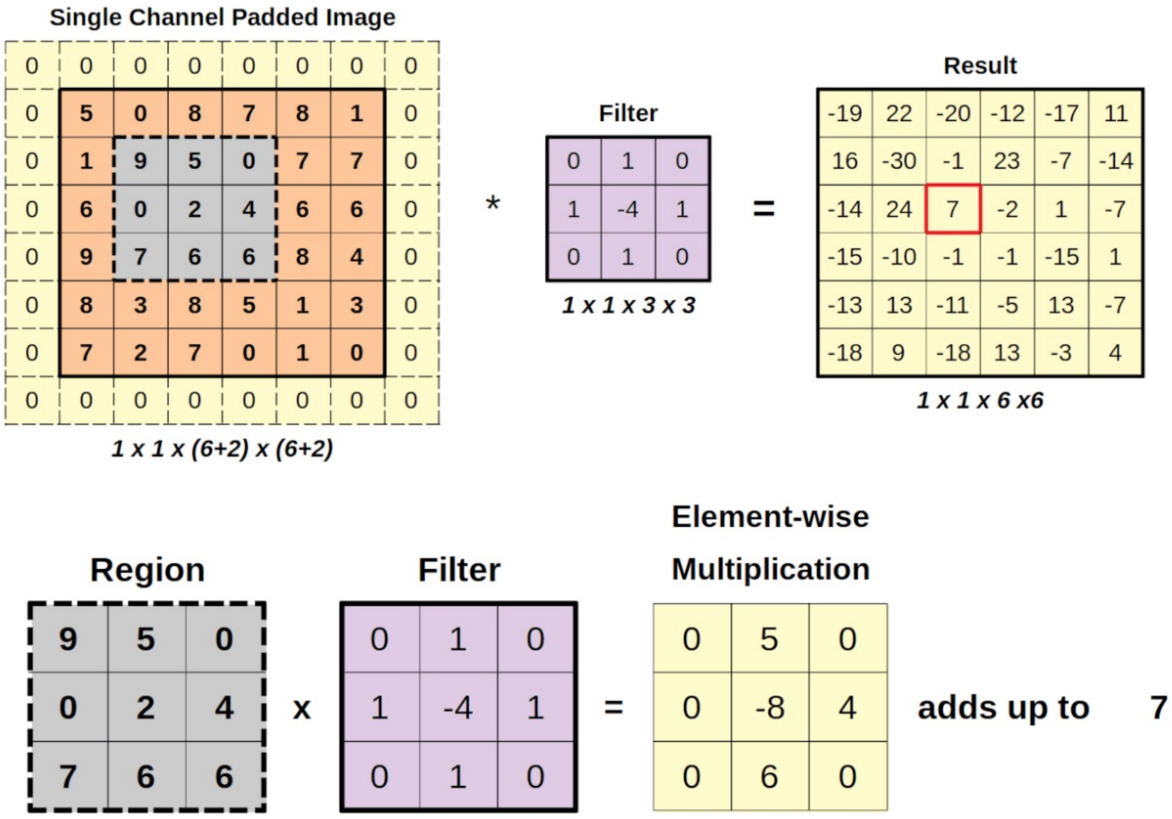

Figure: A convolution filter slides across the image and produces a feature map.

Instead of one massive fully connected weight matrix, CNNs reuse small local filters across the image.

Key Hyperparameters in Convolutional Layers

Three design choices matter immediately:

- **filter size:** how large the receptive field is,
- **stride:** how far the filter moves each step,
- **padding:** whether border information is preserved.

These choices control computational cost, feature resolution, and what structures can be represented.

Pooling and Invariance

Pooling summarizes nearby activations, often using max pooling.

Why Pool?

Pooling reduces spatial resolution, lowers computation, and makes the representation less sensitive to small input shifts.

This is one reason CNNs are so effective for robust image recognition.

CNN Architecture Evolution

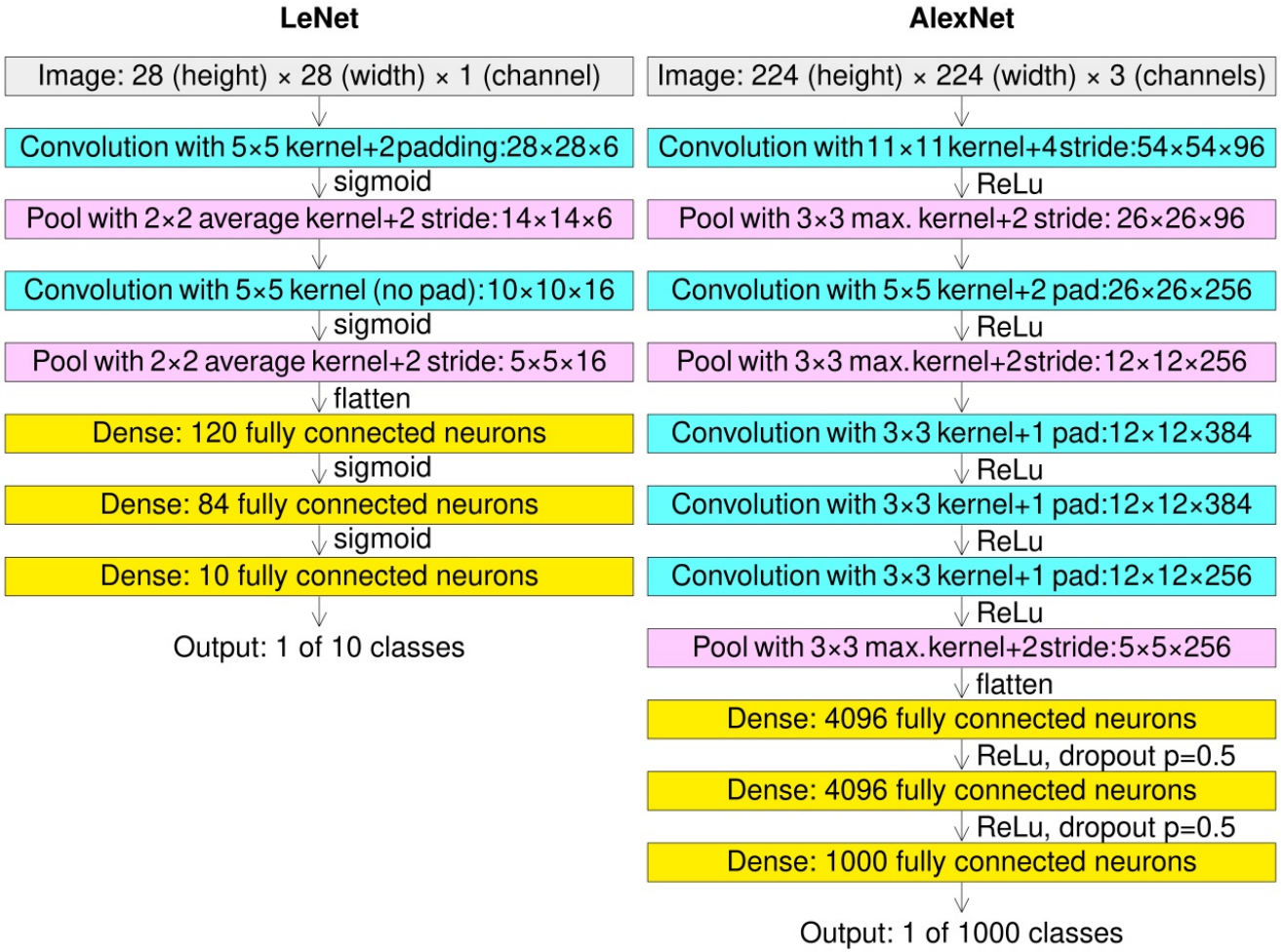

Figure: Representative CNN progression from early to deep architectures.

LeNet to AlexNet to VGG

- **LeNet:** early proof that CNNs worked for digit recognition.
- **AlexNet:** GPU training, ReLU, and depth made CNNs dominant.
- **VGG:** deeper stacks of small filters showed that depth itself is a major source of representational power.

Why ResNet Was a Breakthrough

Once networks became very deep, optimization again became difficult.

Residual Learning

Instead of forcing a block to learn a complete mapping, ResNet asks it to learn a residual correction on top of the identity.

This makes gradient flow far easier in very deep networks.

Residual Block Visualization

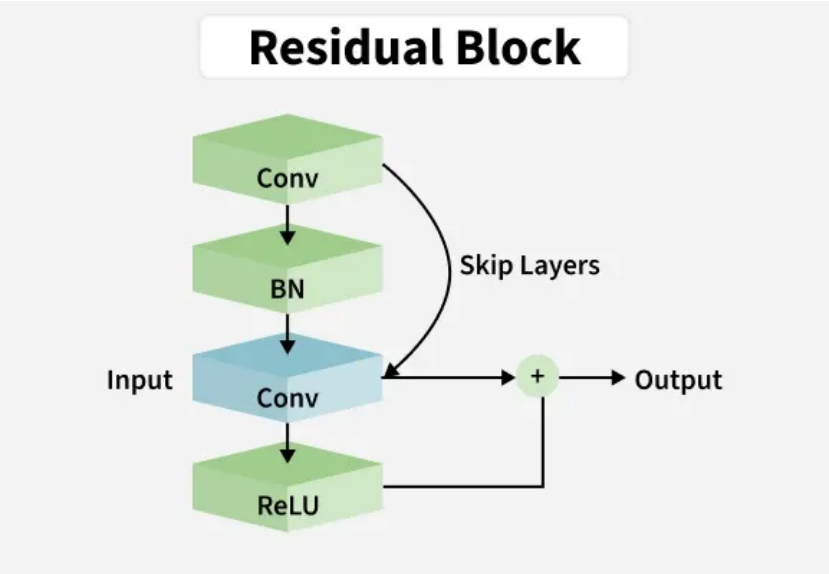

Figure: A residual block adds the input directly to the transformed

Three Reasons CNNs Work So Well

Core CNN Advantages

CNNs use translation equivariance, parameter sharing, and hierarchical feature learning.

- the same detector can work everywhere,
- local filters are parameter-efficient,
- depth builds edges into parts, and parts into objects.

Engineering Applications of CNNs

Where CNNs Fit Naturally

- crack detection in concrete surfaces,
- construction progress monitoring from site images,
- satellite imagery interpretation,
- material and damage-state classification.

What CNNs Do Not Solve Perfectly

CNN Limitation

CNNs are excellent at local spatial structure, but they are less naturally suited to modeling long-range global relationships across a whole image or sequence.

That limitation helps motivate attention-based architectures.

Why Sequence Models Needed a New Idea

Before Transformers, sequence modeling was dominated by RNNs and LSTMs.
Those models process one element at a time, which creates a bottleneck for both computation and memory of long-range dependencies.

RNNs and the Sequential Bottleneck

RNNs update a hidden state recursively:

$$\mathbf{h}_t = f(\mathbf{h}_{t-1}, \mathbf{x}_t).$$

Two Difficulties

Sequential processing limits parallelism, and long-distance information tends to degrade as it must be carried through many time steps.

The Transformer Idea

The Transformer replaces recurrence with self-attention.

Big Shift

Every token can interact directly with every other token in the same layer. The model no longer has to pass information step by step through time.

This is why Transformers scale so well on modern hardware.

Queries, Keys, and Values

Given an input matrix $\mathbf{X}$, the model builds three learned projections:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{Q}, \qquad \mathbf{K} = \mathbf{X}\mathbf{W}^{K}, \qquad \mathbf{V} = \mathbf{X}\mathbf{W}^{V}.$$

Queries ask what this token seeks. Keys describe what each token offers. Values carry the content to be aggregated.

Visualizing Self-Attention

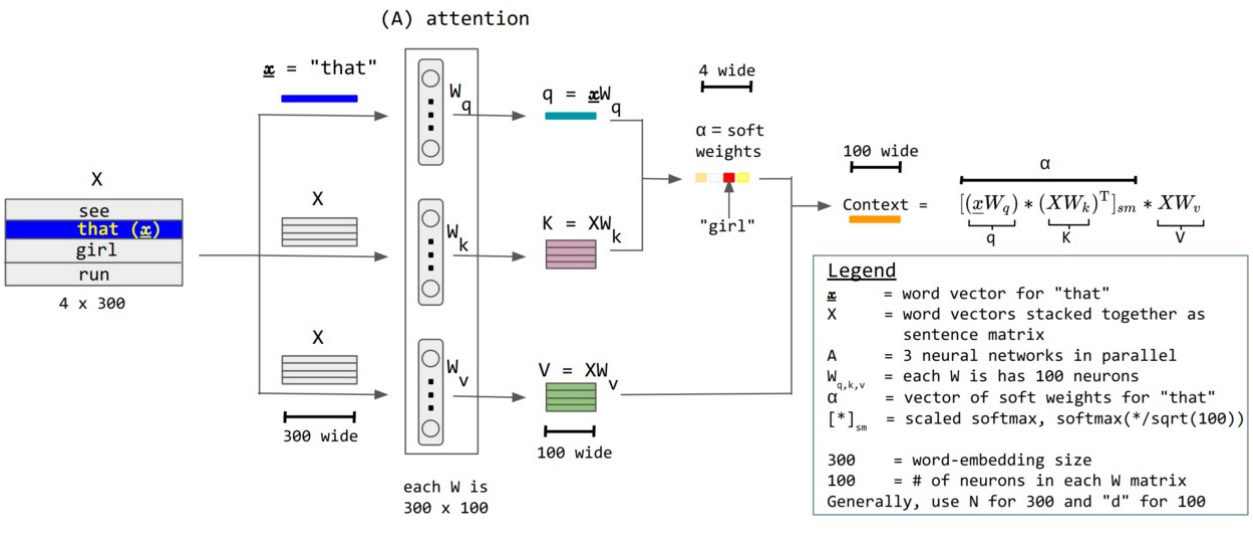

Figure: Query-key-value flow inside a single attention head.

Self-attention converts pairwise relevance into weighted combination of values.

The Attention Equation

The central computation is

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}.$$

The dot products score relevance, the softmax normalizes weights, and the values are blended according to those learned relationships.

Why the Scaling Term Matters

The factor $1/\sqrt{d_k}$ is not cosmetic.

Numerical Role

Without scaling, large dot products can push softmax into saturated regions with tiny gradients, making training less stable.

This is another example of a small numerical idea with a large practical impact.

Multi-Head Attention

A Transformer uses several attention heads in parallel.

- different heads can focus on different relationships,
- one head may track position-like structure,
- another may focus on semantic or syntactic dependency.

This is analogous to multiple learned filters in a CNN.

Why Positional Encoding Is Necessary

Pure attention has no built-in sense of order.

Problem

If we only compare content vectors, the model sees a set of tokens but not their arrangement in sequence.

Positional encoding restores sequence information without recurrence.

The Transformer Block

A standard Transformer block contains:

1. multi-head self-attention,
2. a feedforward network,
3. residual connections,
4. layer normalization.

These blocks are stacked repeatedly to form modern language and multimodal models.

Transformer Architecture at a Glance

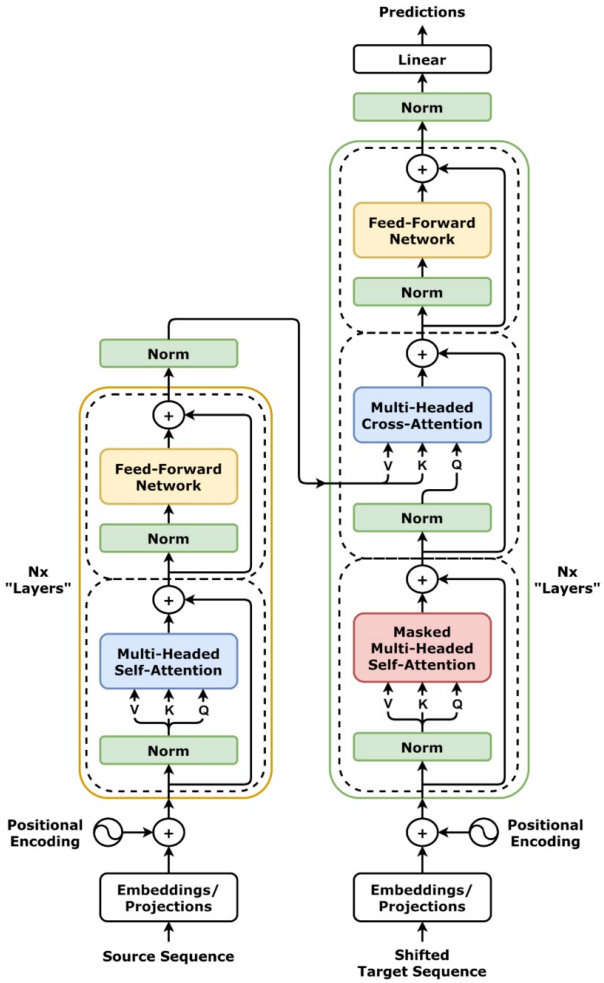

Figure: A representative Transformer architecture.

The exact implementation differs between BERT, GPT, T5,

Encoder-Only, Decoder-Only, Encoder–Decoder

Three common architectural families are:

- **encoder-only:** strong for understanding and classification,
- **decoder-only:** strong for autoregressive generation,
- **encoder–decoder:** strong for input-to-output tasks such as translation.

Modern LLMs are primarily decoder-only Transformers.

Why Transformers Replaced RNNs

Three Main Reasons

Transformers parallelize better, preserve long-range interactions more effectively, and scale more smoothly with data and model size.

That combination made them the architecture behind the LLM era.

The Main Transformer Trade-off

Attention Cost

Standard self-attention scales as $\mathcal{O}(T^2)$ with sequence length. Very long sequences can therefore become computationally expensive.

This motivates work on sparse attention, linear attention, and related efficient-sequence methods.

Engineering Applications of Transformers

Where Transformers Help

- time-series forecasting from structural sensors,
- document understanding for reports and specifications,
- code generation and automation support,
- multimodal fusion of text and vision.

The Intuition Behind Diffusion

Simple Story

Start with real data, gradually add noise until it becomes almost pure randomness, then learn to reverse that process one step at a time.

Generation begins from noise and repeatedly denoises until a coherent sample emerges.

Visualizing the Forward and Reverse Idea

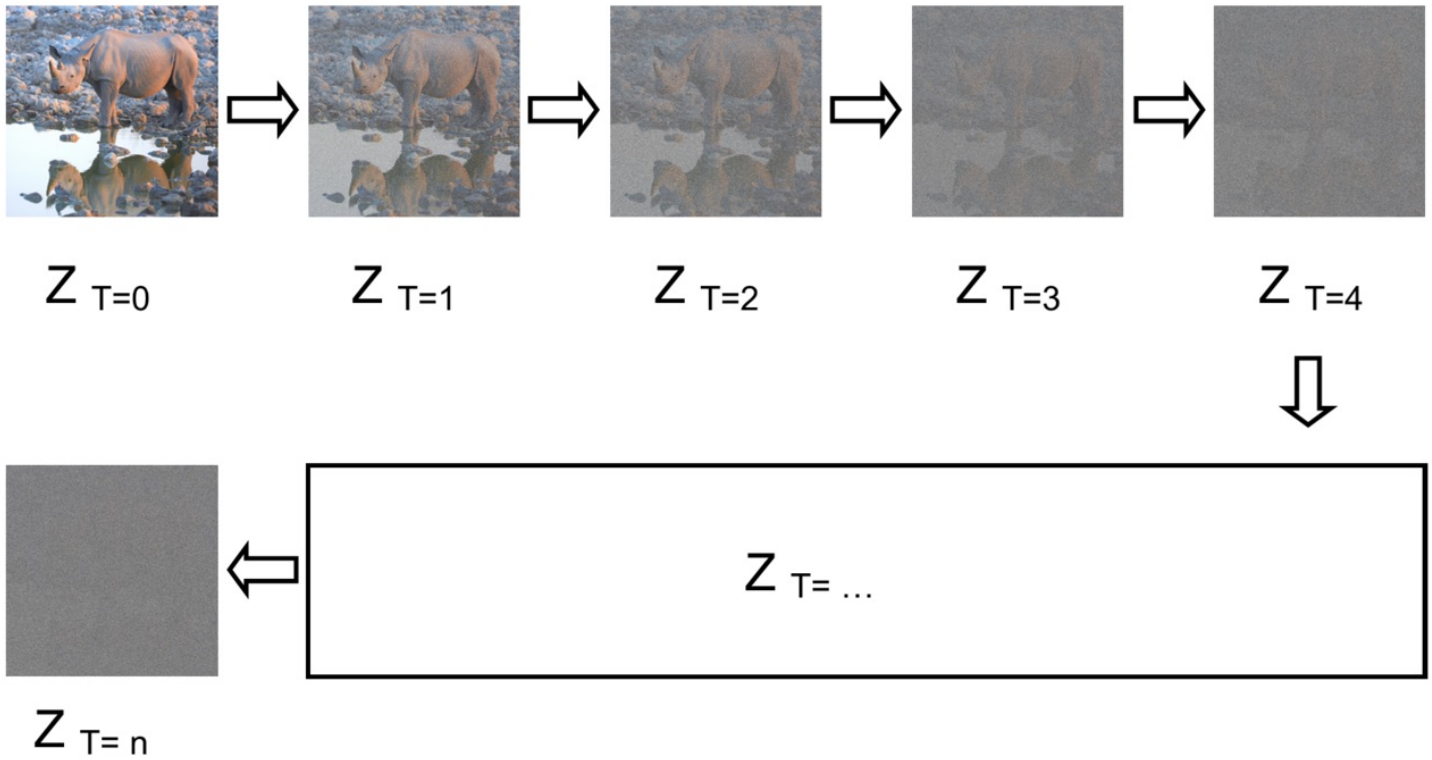

Figure: Forward noising and learned reverse denoising in diffusion models.

This diagram is one of the cleanest ways to understand why diffusion feels conceptually different from GANs.

The Forward Process

The forward diffusion process adds small Gaussian perturbations over many steps:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t \mathbf{I}).$$

Eventually the original structure is overwhelmed and the sample becomes almost indistinguishable from noise.

The Reverse Process

The model then learns a reverse transition

$$p_\theta(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

that removes noise step by step.

Operational Meaning

The model is not directly memorizing images. It is learning how to iteratively refine random noise into structured samples.

Noise Prediction as the Learning Objective

A common training loss is

$$\mathcal{L} = \mathbb{E}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|^2\right].$$

Instead of directly reconstructing the whole data point in one leap, the model learns to estimate the noise component present at each step.

Why Diffusion Models Beat GANs in Many Settings

- training is usually more stable,
- coverage of the data distribution is often better,
- conditioning for text-to-image generation is very effective.

Trade-off

The cost is slower generation because many denoising steps are required.

Engineering Applications of Diffusion Models

Potential Uses in Engineering

- synthetic image generation for rare damage cases,
- concept rendering and design ideation,
- scenario visualization for hazards or infrastructure planning,
- data augmentation when labeled data are limited.

CNNs, Transformers, and Diffusion: Different Strengths

These models are not interchangeable.

High-Level Positioning

CNNs are strongest when locality and spatial regularity dominate. Transformers are strongest when global relationships and sequences dominate. Diffusion models are strongest when high-quality generation is the priority.

Comparison by Inductive Bias

- CNNs assume spatial locality matters.
- Transformers assume flexible pairwise interaction matters.
- Diffusion assumes generation can be learned as gradual refinement.

Engineering Lesson

Good models are not only powerful. They are well matched to the geometry and structure of the problem.

Modern Systems Often Combine These Ideas

The current AI landscape is full of hybrids:

- vision Transformers treat image patches as tokens,
- diffusion models often use U-Net-style CNN backbones,
- multimodal systems combine visual encoders and language decoders.

The boundaries between architecture families are becoming increasingly porous.

What Makes These Architectures Generative?

Architecture versus Modeling Goal

CNNs, Transformers, and diffusion models are not automatically generative. They become generative when they are trained to produce new samples rather than only classify or predict from existing inputs.

- CNNs become generative inside decoders, U-Nets, and image generators.
- Transformers become generative when they generate sequences token by token.
- Diffusion models are generative because they learn a sampling process from noise to data.

Old Generative Models versus New Generative AI

In classical machine learning, a model was called **generative** if it explicitly modeled how data were produced.

- Examples: Gaussian mixture models, hidden Markov models, Bayesian latent-variable models.
- Modern deep generative AI keeps that goal, but uses neural networks to synthesize complex high-dimensional outputs directly.

The New Emphasis

"Generative" now means not only describing a probability structure, but actually constructing plausible new images, text, layouts, signals, or scenarios from learned data structure.

From Topic 13 to Topic 14

- A **discriminative** model asks: given this input, what label, class, or value should I predict?
- A **generative** model asks: what plausible sample could have come from this data-generating process?
- For engineers, this means AI can support not only classification and forecasting, but also synthesis, reconstruction, and design exploration.

In Topic 14, we turn explicitly to that generative perspective through GANs, VAEs, and diffusion models in practice.

Key Takeaways

- Deep learning succeeded because training became numerically manageable.
- CNNs are the natural choice for many vision problems.
- Transformers are the backbone of modern language models.
- Diffusion models are central to modern high-quality generation.
- Engineering judgment still matters: architecture choice must follow problem structure.